

Explainable AI-based intrusion detection in IoT systems

Sarah Bin hulayyil ^{a,b}, Shancang Li ^{a,*}, Neetesh Saxena ^a

^a School of Computer Science and Informatics, Cardiff University, Cardiff, CF24 4AG, UK

^b College of Applied Studies and Community Service, King Saud University, Riyadh, 11451, Kingdom of Saudi Arabia

ARTICLE INFO

Keywords:

IoT

XAI

Attack detection

Intelligent intrusion detection

ABSTRACT

The Internet of Things (IoT) systems are highly vulnerable to cyber attacks due to limited and/or default security measurements. Machine learning (ML) techniques bring a powerful weapon against the insecurities of IoT systems, such as intelligent intrusion detection systems (IDSs), vulnerability/threats detection, and behavioral analysis. ML-based IDSs offer a significant improvement in IoT security, but they also bring technical challenges, e.g., false positives, evolving attacks, data quality and bias, explainability and transparency, etc. Explainable Artificial Intelligence (XAI) can address these challenges by offering interpretable and comprehensible insights into the ML-based IDS decision-making process. A novel framework for an explainable IDS-based vulnerable IoT devices related to the Ripple20 vulnerability and its associated attacks. The framework integrates ML classifiers and XAI techniques to provide comprehensive and interpretable explanations for the IDS decisions. We evaluated this framework on various datasets, including a dataset collected from the labs and other public datasets, using binary and multi-classification models. The experimental results demonstrate the efficiency and accuracy of the framework in detecting and categorizing IoT vulnerabilities. The framework also offers benefits over conventional IDS systems, such as facilitating comprehension and confidence among security experts, enhancing the precision and efficiency of the detection procedure, and adapting to the dynamic IoT environment.

1. Introduction

Over the past few years, the IoT has experienced a notable surge in popularity, establishing itself as an indispensable component of our daily existence [1]. The exponential expansion of IoT devices has created many security vulnerabilities, making these devices highly susceptible to various malicious attacks [2]. Detecting and mitigating these vulnerabilities is of the utmost importance in safeguarding the security and privacy of IoT networks.

Identifying vulnerabilities within the IoT is important, as it pinpoints potential security threats and facilitates the implementation of proactive security measures to prevent potential attacks [3]. Traditional intrusion detection systems (IDS) have gained significant usage [4]; however, they often need to improve interpretability, which is the ability to understand and clarify the decision-making process of a machine learning model, and do not provide substantial insights into the underlying rationales behind their determinations [5]. This limitation makes it extremely difficult for cyber security experts to identify, address, and mitigate vulnerabilities efficiently and promptly.

To tackle the issue of identifying vulnerabilities in IoT systems, this work focuses on the development of an eXplainable Intrusion Detection (X-IDS) framework on the network. Our primary focus is on the Ripple20 vulnerability, an assemblage of zero-day vulnerabilities recently discovered within the networking stacks of prevalent IoT devices.

* Corresponding author.

E-mail address: Shancang.li@ieee.org (S. Li).

<https://doi.org/10.1016/j.iot.2025.101589>

Received 18 October 2024; Received in revised form 20 January 2025; Accepted 21 March 2025

Available online 4 April 2025

2542-6605/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

The Ripple20 vulnerabilities consist of 19 vulnerabilities that reside in the Treck TCP/IP software library, which is used by many IoT devices. In June 2020, JSOF published details of these vulnerabilities, stating that they can be exploited in different manners: Internet-facing Devices: IoT devices that are directly accessible to the internet are at increased risk [6]. Exploitation of these vulnerabilities remotely can lead to control of the affected device. Internal network attacks: after an attacker has successfully gained an initial foothold within an internal network, Ripple20 vulnerabilities can be utilized to attack specific devices within that internal network. Supply Chain Attacks: since the Treck TCP/IP library is widely used, devices in various industries are impacted. This includes medical devices, industrial control systems, and printers. Long-Term Exploitation: attackers can stay in systems for years by hiding malicious codes within devices [6].

As ML is used to detect and identify IoT attacks and threats aiming to enhance IoT security, many of them have a major challenge as the lack of interpretability. Traditional IDS often provides binary output, simply classifying traffic as an attack or normal, without explaining how the decision was made. This opacity makes it difficult for cybersecurity professionals to understand why the IDS flagged an event and hinders their ability to implement appropriate mitigation strategies, especially in time-constrained environments.

Acquiring labeled data for IoT vulnerabilities is another key challenge: the dynamic nature of IoT networks, makes it difficult to capture representative data, and the scarcity of real-world attack data, limits the training of robust models. This limited data significantly hinders the development and evaluation of effective vulnerability detection systems [7].

Rapidly evolving IoT landscape: the IoT landscape is undergoing rapid evolution, characterized by the continuous emergence of novel devices and vulnerabilities. The adaptability of vulnerability detection systems is of the utmost importance to ensure the provision of accurate and current detection capabilities.

The main objective is to showcase the efficacy of our approach in identifying and categorizing IoT devices that are susceptible to the Ripple20 vulnerability as a use case. By implementing our framework, we aim to provide comprehensive explanations for the decisions made by the IDS, enhancing its overall effectiveness. Ripple20 vulnerability is used as a use case due to the high potential risks that could be caused by the set of 19 zero-day vulnerabilities. To address the above challenges, this work focuses on an explainable IDS framework and vulnerability analysis using machine learning models. The main contributions are summarized as follows.

- Acquiring a comprehensive dataset that encompasses real-time data from our smart home laboratory (CUSmartHome). This dataset covers various IoT devices, including vulnerable and non-vulnerable devices. The process involves extracting and engineering pertinent features from the data, which are then used to train and evaluate our IDS framework.
- To train and evaluate the proposed IDS framework, the CUSmartHome dataset was then processed to extract and engineer relevant features that aid in identifying the 19 known zero-day Ripple20 vulnerabilities and their associated attacks. The performance of our framework was assessed using metrics such as accuracy, precision, recall, and F1-score.
- A novel explainable IDS framework is developed that leverages the network infrastructure to effectively identify and categorize susceptible IoT devices. The framework seamlessly integrates machine learning ML classifiers by combining SHAP and causal inference to ensure robust, interpretable, and actionable insights on analyzing network traffic data and classifying devices according to their susceptibility to Ripple20 vulnerabilities and associated attacks.
- Conducting a comprehensive assessment of our proposed framework by thoroughly testing it on two supplementary datasets namely the widely recognized IoT23 dataset and an IoT environment dataset. Including these datasets allowed for a more comprehensive assessment of our framework's performance and capacity to adapt to various IoT network scenarios.

The remaining of this paper is structured as follows. Section 2 provides a review of relevant research and background information. Section 3 details the methodology, including the proposed approach and the proposed framework architecture. Section 4 presents the results and evaluation of the proposed model. Finally, Section 5 concludes the work by summarizing the key findings and presenting potential future work.

2. Related works

2.1. XAI within IoT

The onset of IoT has caused a paradigm change in numerous sectors, encompassing intelligent cities, healthcare, and transportation [8]. Nevertheless, IoT networks' pervasive connectivity and seamless data exchange have undeniably ushered in many cybersecurity challenges. In response to these concerns, cybersecurity has developed IDSs as effective security mechanisms. DL algorithms, particularly Deep Neural Networks (DNNs), have demonstrated considerable potential in enhancing the detection rate of intrusions related to the IoT.

Despite their undeniable capabilities, DL-based models often exhibit a complexity that poses challenges in terms of interpretability. This is particularly true for cyber security experts and executive staff within companies. The absence of interpretability poses a significant obstacle for users, impeding their trust and ability to optimize the decisions made by ML models. The field of XAI has recently emerged within the domain of AI. Its primary objective is to develop techniques that facilitate the interpretation and comprehension of decisions made by ML models. XAI is crucial to improve the interpretability of ML-based IDSs. This, in turn, empowers users to place their trust in the system and make informed decisions while optimizing their outcomes using the output generated by ML models.

A groundbreaking framework was introduced in a recent publication [7]. This framework leverages XAI to offer comprehensive explanations for crucial decisions made by the DL algorithm in IDS on the IoT. The framework utilizes an innovative IDS specifically designed for IoT networks. This IDS incorporates a cutting-edge Deep Neural Network (DNN) model, which enhances its ability to detect intrusions specifically targeted towards IoT devices. Moreover, the authors adeptly integrated three prominent XAI techniques, specifically RuleFit, Local Interpretable Model-Agnostic Explanations (LIME), and Shapley Additive explanations (SHAP), to comprehensively explain at the local and global levels. The explanations provided at the local level are centered on a singular or specific output of the DL system. This enables users to gain a comprehensive understanding of the precise rationale behind a given decision. In contrast, global explanations serve the purpose of inferring the key features or factors that play a significant role in the overall decision-making process of the DL model. The framework enhances the interpretation of decisions made by the IDS by effectively combining various types of explanations. However, this framework provides interpretations only of the DNN model, this could work well and yield good results in some cases but produce unexpected results in other cases, so the beneficial use of this framework is limited to the result from the DNN model and cannot be generalized to uses with cases that do not yield high results.

Further, Zolanvari et al. [9] introduced a groundbreaking XAI model, TRUST (Transparency Based on Statistical Theory). This model aims to offer explanations in numerical applications, presenting a fresh perspective on the field. This paper focuses on utilizing TRUST within the field of IIoT security while also evaluating its performance and benefits compared to alternative XAI models, which achieved an average success rate of 98% in explaining new samples. In addition, evaluated the advantages of TRUST over LIME in terms of performance, speed, and explainability.

Tabassum et al. [10] discussed the use of XAI to explain the categorization of IoT network attacks and increase transparency and reliability in the ML and DL models. It applied various ML models to detect IoT network attacks and then achieved a success rate of 96% using XAI approaches such as SHAP, LIME, and ELI5 to interpret and explain the categorization of IoT network attacks. However, this paper did not use real-time data and applied XAI with limited explanation and results.

Moreover, a recent study [11] presented a framework that combined XAI, DL, and evolutionary computation to solve IoT applications. It converts IoT data into an image database using the Gamian angular field and trains the images using VGG16. XAI technology and hyperparameter optimization were introduced to analyze the impact of input values and understand the weights of the DL model. The proposed approach shows improved efficiency compared to the baseline approaches in terms of both run time and accuracy. However, converting IoT data into images could be costly and can challenge the generalizability and scalability of the proposed approach to different types of IoT applications or datasets.

Another study discussed an XAI-based approach for text summarization of social IoT-based content, but it does not specifically mention the combination of XAI and IoT [12]. It proposes an XAI-based reinforcement learning approach for text summarization of social IoT-based content, aiming to condense a given text while maintaining coherence and readability. Additionally, the framework combines reinforcement learning with a ground-truth-dependent text summarization model to improve sentence coherence and content quality in summary with good results. However, the standard supervised training has substantial data costs [12].

The latest study addressing the issue of the traditional IDS, focuses on detecting Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks in a real-time situation. It proposed a hybrid framework that integrated the KNN algorithm with dense neural networks. When tested using the MQTT-IoT-IDS2020 dataset, the results demonstrated that the system performs better than existing techniques in terms of accuracy, precision, and recall rates for identifying and categorizing DoS/DDoS attacks in real-time scenarios. This novel method demonstrates how hybrid models can be used to obtain some of the limitations of traditional IDS [13].

2.2. Scoping review of the XAI within IoT

On the other hand, some survey papers should be discussed to determine the limitations of using XAI in IoT security. A survey paper by Adadi and Berrada [14] presents a comprehensive examination of XAI. The authors engaged in a thoughtful conversation regarding the absence of transparency observed in AI-driven systems, emphasizing the necessity for XAI to enhance trust and promote transparency. This paper comprehensively reviews current approaches and techniques in XAI. It analyzes emerging trends within the domain and offers insights into significant potential future directions. The survey is a valuable resource for researchers and practitioners interested in the field of XAI.

The significance of explainability in AI is underscored in the work by David Gunning et al. [15]. The paper highlights users' need to comprehensively understand, place confidence in, and effectively manage AI applications. The significance of explainability in ensuring AI's widespread adoption and user trust cannot be overstated.

Jagatheesaperumal et al. [16] delves into the IoT realm and examines the use of XAI. It presents a meticulous and all-encompassing survey of various XAI frameworks, explaining their particular features and relevance to facilitate IoT applications. The paper explores the utilization of XAI services across a range of IoT applications, including but not limited to bolstering security measures, facilitating improvements in the Internet of Medical Things (IoMT), optimizing operations in the Industrial IoT (IIoT) sector, and enhancing functionality in the Internet of City Things. The paper examines the options for the implementation of XAI models within IoT systems, the latest advancements in edge XAI architectures, and the integration of 6G communication services to support IoT applications.

In the realm of IoT, Kok et al.. [17] have contributed another noteworthy survey paper that delves into the application of XAI models. This paper comprehensively analyzes recent studies that employ XAI models in IoT applications. Systematically categorizes these studies based on their methodology and application domains, while shedding light on the challenges and unresolved matters within this field. Furthermore, the paper presents a compelling analysis of potential avenues for future research in XAI within the IoT context.

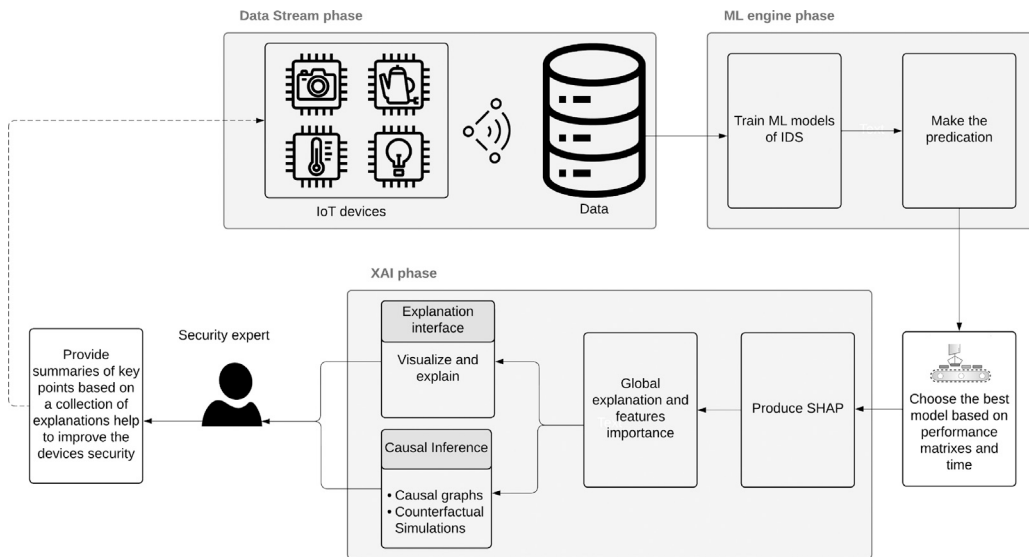


Fig. 1. The XIDS architecture.

Table 1

Datasets for training/validation of the framework (T_b -number of balanced training).

Dataset	No. of packets	No. of T_b	Classification type
CUSmartHome	305 238	341 202	Binary classification
Sub-IoT23	1 760 865	2 093 798	Binary classification
IoT-e-dataset	1 354 555	3 582 104	Multi-classification

Moustafa et al. [18] explored the realm of applying XAI in the context of intrusion detection for IoT networks. The paper meticulously examines various XAI techniques employed for anomaly-based intrusion detection in IoT networks, focusing on leveraging DL models to facilitate interpreting and explaining security events. Similarly, Van der Velden et al. [19] demonstrated the use of XAI in DL based on medical image analysis. They thoroughly examined the challenges in this context and proposed viable solutions to address them. Additionally, the paper sheds light on potential avenues for future research in this domain.

Our work focuses on solving the limitations of the previous studies which are the interpretability behind the decision of the XAI to detect threats that could affect the IoT security. In addition, the flexibility of extending this framework to include more ML models that are used to enhance IoT security. It presents an innovative framework that leverages XAI methodologies to identify vulnerabilities in IoT networks and comprehensively explain the decisions made by ML models. This approach attempts to address the need for greater interpretability in extant ML-based IDS and augments the comprehension of the rationales underlying the model's determinations.

The contribution of this paper is noteworthy, as it has the potential to substantially enhance the cyber security of IoT networks. Through the precise identification of vulnerabilities and the provision of comprehensive explanations for the decisions made, this framework can help cybersecurity professionals make well-informed judgments and efficiently mitigate IoT vulnerabilities. It fortifies confidence in the IoT system and empowers users to optimize their outcomes. Furthermore, the framework has the potential to facilitate the advancement of AI systems that are characterized by enhanced transparency and reliability, thereby fostering the extensive acceptance and utilization of AI applications.

3. Proposed approach

In this work, we set out our Explainable Intrusion Detection Network Base for IoT vulnerability framework, as shown in Fig. 1. The framework includes three key phases: first, the data stream, which collected data from different IoT devices and then pre-processed the data by feature engineering and feature selection to determine which features are most relevant. Second, vulnerability detection and performance selection; in this step, the data will be trained in all ML classifiers, and then the best model will be selected based on the performance and time to pass it to the next phase. Third, the XAI phase; once the best classifier was chosen, its choices were examined using the SHAP technique combined with casual inference, which offers a thorough explanation of features from a variety of perspectives and how they contribute to the classification task. All of these phases result in beneficial explanations for cybersecurity experts that could be used to improve IoT security.

We selected the Ripple20 vulnerability and its attacks as a use case and then used three datasets. The first dataset was collected from our Smart Home Lab from 12 IoT devices, six of which have the Ripple20 vulnerability, while the other six are

free of this vulnerability and named these data CUSmartHome. The other two datasets are public. A subset of the IoT23 dataset contains packages consisting of DDoS attacks resulting from the Ripple20 vulnerability [20]. Finally, the IoT environment dataset consists of normal and port scan packets, OS and service detection packets, and flooding attack packets, all related to the Ripple20 vulnerability [21]. The data type used for all data is *pcap* files collected using *Wireshark*. Table 1 shows the number of packets that are used and the purpose of classification in each dataset. The next subsections explain the XIDS in detail.

3.1. The XIDS architecture

The proposed framework comprises three primary stages: the data stream phase, the ML engine phase, and the XAI phase. During the data stream phase, real-time data was collected from the IoT network and processed to extract pertinent features associated with the Ripple20 vulnerability. The extracted features encompassed a range of elements, including size-based features, DNS names, time-based features, source and destination features, and protocol-related features.

We trained and evaluated numerous ML classifiers using the extracted features in the ML engine's subsequent phase. Our analysis examined a variety of classifiers, including DNN, Gaussian Naive Bayes (GNB), Bernoulli Naive Bayes (BNB), Logistic Regression (LR), Stochastic Gradient Descent Classifier (SGD), Random Forest Classifier (RF), Extreme Gradient Boosting Classifier (EGB), Quadratic Discriminant Analysis (QDA), Support Vector Classifier (SV) and Decision Tree Classifier (DT). We used these classification models because they are existing models that could be explained. In this stage, the performance of each classifier is evaluated using metrics such as accuracy, precision, recall, and F_1 -score.

$$y = f(x; \theta) \quad (1)$$

For binary classification, we have

$$R(h^*) = \inf_{h(\cdot)} R(h) \quad (2)$$

in which the infimum is taken over all classifiers, e.g., functions from $\mathcal{X}$ to $\{0, 1\}$.

For example, a Bayes optimal classifier h^* , can be described by

$$h^*(x) = \arg \min_{c \in \mathcal{Y}} R(c | x) \quad (3)$$

In the XAI phase, a comprehensive analysis was conducted on the features and decisions made by the top-performing ML classifier, employing the SHAP method (Shapley additive exPlanations). The SHAP offers comprehensive explanations on a global scale, shedding light on the features involved in a classification task and their significance [22]. The explanations are presented in a dedicated interface designed to enhance the understanding of the decisions made by security experts within the IDS system.

The Shapley values are defined by

$$g(z') = \phi_0 + \sum_{j=1}^M \phi_j z'_j, \quad (4)$$

in which g is the explanation model, $z' \in \{0, 1\}^M$ is the coalition vector, M is the maximum size of the coalition and ϕ_j is the feature j attribution, the Shapley values.

To calculate the Shapley values, we may use

$$g(x') = \phi_0 + \sum_{j=1}^M \phi_j. \quad (5)$$

The local accuracy can be approximated by

$$\hat{f}(x) = g(x') = \phi_0 + \sum_{j=1}^M \phi_j x'_j \quad (6)$$

The consistency can be defined by

$$\hat{f}'_x(z') - \hat{f}'_x(z'_{ij}) \geq \hat{f}_x(z') - \hat{f}_x(z'_{ij}) \quad (7)$$

in which for input $z' \in \{0, 1\}^M$, we have

$$\phi_j(\hat{f}', x) \geq \phi_j(\hat{f}, x) \quad (8)$$

3.1.1. Data stream phase

The data stream phase entails acquiring real-time data from IoT systems, followed by its subsequent processing to extract salient features closely linked to the Ripple20 vulnerability. The extracted features encompass diverse elements, consisting of *size-based features*, *DNS names*, *time-based features*, *source and destination features*, and *protocol-related features*.

Data preprocessing is implemented using *feature selections* and *feature engineering*. The selected features that were related to our case of Ripple20 vulnerability, and their consequences such as DoS attacks, are shown in Table 2. Table 3 provides the notation used.

Table 2
The Ripple20 vulnerability associated features.

Related features	Feature name	Description
Size-based features	Packet length	Length of packet (bytes)
	Hheader length	Size of packet header
	Payload length	Size of the payload (data)
DNS names	DNS name	DNS domain name
Time-based features	Packet flow duration	Window time/packet
Source and destination	Source IP	IP address of the sender
	Destination IP	IP address of the receiver
	Source port	Source port number
	Destination port	Destination port number
Protocol-related features	Protocol	Protocol used
	SYN_flag	SYN packet
	ACK_flag	ACK packet
	FIN_flag	FIN packet

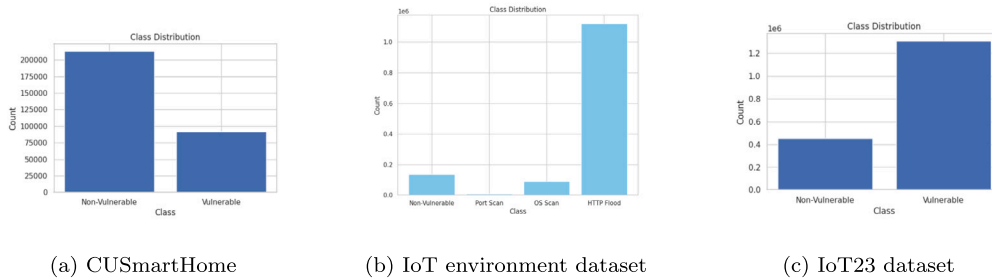


Fig. 2. The number of classes in each dataset.

To ensure data quality and the datasets are clean, interpretable, and robust, thus reducing biases and inconsistencies in the datasets, we implement a systematic preprocessing and feature engineering pipeline: **Feature Extraction:** Extracted packet-level ‘packet-length, protocol, src-ip, dst-ip and flow-level flow-duration features from raw PCAP files using Scapy. For an improved analysis, additional properties such as syn-flag, ack-flag, and dns-name were generated. **Handling Missing Values:** Filled in missing values in critical features such as dns-name, src-port, flow-duration with default values, further combating data inconsistency and improving coherence. **Protocol Mapping:** The mapped numerical protocol identified was mapped and provided human-readable protocol names such as TCP and UDP for improved interpretability. **Bias Mitigation:** Balanced the label representation by processing normal and vulnerable traffic data from each dataset. In addition, encoding categorical variables such as src-ip, dst-ip, etc. to ensure that all the features are present in a consistent way across all the datasets. **Combining Data:** Divide feature definitions into columns of DataFrames, then store the cleaned data in CSV format for use in subsequent modeling.

3.1.2. ML engine phase

During the ML engine phase, we used 80% of each dataset to train the classifier and 20% to test the classifier. A diverse array of ML classifiers were meticulously trained and rigorously evaluated, utilizing the carefully extracted features, including DNN, Gaussian NB, Bernoulli NB, Logistic Regression, SGD Classifier, Random Forest Classifier, XGB Classifier, QDA, SVC and Decision Tree Classifier. The main reason for selecting different ML classifiers is to allow the framework to train and choose the best performance classifier. That would make the results better for the reliable and accurate model rather than sticking to one classifier, as different data could be compatible with one model rather than others.

As shown in Fig. 2, the traffic data of the network is often unbalanced. In order to avoid the model being biased towards the majority class, we created a balance for training data only, and thus not performing well on the minority class. Creating balance for the entire dataset, including the test or validation data, can risk introducing bias into the model evaluation. It may perform well in the testing set, but this performance may not be generalized to real-world scenarios where the data distribution is unbalanced. Thus, it was important to ensure that the model was trained to make accurate predictions in real-world scenarios, even when class imbalances are present. The approach used to balance the data was oversampling because this method usually achieves better performance and results than other techniques [23].

Oversampling is implemented as part of our pre-processing pipeline in order to address issues related to unbalanced datasets and, therefore, to allow the classifier to more accurately identify and classify the features belonging to all classes. This approach increases the number of samples in the minority class by duplicating random samples. To prevent any form of data leakage and maintain the efficacy of our model evaluation, oversampling is applied only to the training set within each fold of the cross-validation process. Thus, a fair performance of the model is obtained and provides a more robust and generalized model.

Table 3
Initialize variables.

Symbols	Definition
$M[m]$	The classification models
X	The dataset
MA	Model accuracy
BA	Best accuracy
MT	Model time
BT	Best time
MS	Model score
BS	Best score
BM	Best model

This work rigorously evaluated each classifier using various performance metrics, including accuracy, precision, recall, and F_1 -score, applied to both the training and testing datasets. Since our primary goal was to enhance IoT security, we also considered the training time of each classifier, acknowledging the time constraints inherent in IoT environments. This comprehensive evaluation allowed us to identify the classifier that exhibited the best overall performance, paving the way for its selection for further analysis.

This phase focused on interpretable models compatible with SHAP. However, the framework is designed to accommodate additional models in the future as more interpretability techniques emerge. Furthermore, the results of all processes within this step can be conveniently printed and reported, allowing clear monitoring by security experts. In particular, the automatic output from this step is unique, identifying the best performing model based on both its training and testing accuracy, as well as its training time. This optimal model will then be passed to the next stage for further analysis.

Algorithm 1: Model Selection Algorithm

Input: Dataset X , model set M , $m = \emptyset$, $i = 1 \dots N$
Output: m_{output}
Initialize: $m_o = m_1$, $T_1 = T_o = 0$, $S_1 = S_o = 0$, $A_1 = A_o = 0$, $i = 1$;
while $i \leq N$ **do**
 ModelTrain(m_i , X_{train} , Y_{train});
 (P_i , t_i , acc_i) = $m.predict(X_{test})$;
 $A_i = avg(m[train], m[test])$
 if $A_i \geq A_o$ **and** $S_i \geq S_o$ **and** $T_i < T_o$ **then**
 $A_o = A_i$;
 $T_o = T_i$;
 $S_o = S_i$;
 $m_o = m_i$;
 $i \leftarrow i + 1$;
return m_o

The algorithm **returns** the best model m_o , which has the highest accuracy and score within the shortest training time compared to all other models evaluated.

3.1.3. XAI phase

The classifiers with the highest performance were chosen and then SHAP combined with causal inference was used to analyze its decision-making processes, which offers a comprehensive explanation of both the overall impact of features and their individual importance in influencing the classifier's predictions. The aim of combining SHAP within causal inference is to improve explainability and decision-making by showing that the importance of the feature indicates a true causal relationship and not a mere correlation. Offering useful knowledge through counterfactual and intervention exploration. Improving the explanations of feature importance by reducing the impact of confounding variables. Enhance the transferability and robustness of the insights obtained for different contexts. Improve the quality of decision making by focusing on causal and actionable factors. In general, it helps to obtain interpretable and reliable model output. We used two main uses of causal inference; firstly, we passed the top five important features to explain the model's decision based on the relationship between these features and figured Causal Graph. Secondly, using counterfactual simulations to assess the causal effects of changes in specific features.

SHAP was used to explain the rationales behind the determinations rendered by the ML models because it offers a methodical approach to elucidate the results generated by these models. This is achieved by assigning a numerical value to each input feature, thereby denoting the feature's contribution to the overall output of the model. This is useful to ascertain the importance of features in the model's predictions. The elucidations are presented via an explanatory interface, which assists security experts in comprehending the determinations made by the IDS and the interrelations among the various features.

The simplified mathematical explanation of SHAP in the context of explaining the output of an ML model for a specific instance can be presented as follows. Let m be an ML model that takes a feature vector X as input and produces an output $m(X)$. The

goal is to explain the output $m(X)$ in terms of the contributions of individual features. SHAP values are essential for improving the interpretability of machine learning models. Individual features are given scores by SHAP values, which show how much each feature contributed to the model's output and assist in understanding how models make decisions. In addition, they offer a framework that is independent of the model, enabling a consistent interpretation of various machine learning models [24]. SHAP values succeed in finding important features because they satisfy key properties, interpretability which the contributions of all features sum to the total prediction, allowing deniable attribution of the importance to all features. Due to symmetry, features that provide evenly will receive identical SHAP values indicating their equivalent relevance. Any feature that does not contribute to the prediction gets a SHAP value of zero, which means that the feature is not important. Therefore, they compute the average marginal contribution of a feature over all possible combinations of features such that a fair distribution of importance is achieved. As a result, they are effective in determining the importance of features because they take into account interactions between features and present a holistic metric of the influence of each feature.

Mathematically, the SHAP value for a feature i is defined as the average contribution of feature i across all possible orders in which the features could arrive. The SHAP value for feature i can be calculated using Eq. (9)

$$\phi_i(m) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! \cdot (|N| - |S| - 1)!}{|N|!} [m(S \cup \{i\}) - m(S)] \quad (9)$$

in which

- N is the set of all the features.
- S is a subset of N that does not contain feature i .
- $m(S \cup \{i\})$ is the output of the model when the feature i is added to the subset S .
- $m(S)$ is the output of the model for the subset S without feature i .
- $|S|$ is the number of features in subset S .

In addition, by adding together all of the feature input values and averaging each column's particular feature, SHAP analyzes global interpretations. Thus, the following could be used to get SHAP's explanation for the most important feature:

Assuming that we have calculated the average SHAP value S_{A_j} for each feature j across all n data samples using Eq. (10)

$$S_{A_j} = \frac{1}{n} \sum_{i=1}^n \phi_{ij} \quad (10)$$

in which ϕ_{ij} is the Shap value for the j th feature in the i th data sample.

To identify the most important features globally, we ranked the features on the basis of the absolute values of their average SHAP values. The ranking was done by sorting the features in descending order of their absolute average SHAP values. To rank the elements of a vector $S_{A_j}, j = 1, \dots, n$, we use

$$R(j) = \text{rank}(S_{A_j}), j = 1, 2, \dots, n \quad (11)$$

The rank of S_{A_j} can be computed by

$$R(j) = 1 + \sum_{i=1}^n \mathbb{I}(S_{A_i} < S_{A_j}) \quad (12)$$

in which $\mathbb{I}(\text{condition})$ is the indicator function, which is 1 if the condition is true, and 0 otherwise.

This means that the feature j that maximizes the absolute value of its average SHAP value. Features with larger absolute average SHAP values are considered more globally important because they, on average, have a more significant impact on the model output across all data samples [22,25].

By integrating casual inference within SHAP, to achieve the best understanding that helps the expert to improve IoT security. From Eq. (10), the absolute average SHAP values S_{A_j} are used to rank the importance of features globally. The top k important features are selected as follows:

$$F_k = \{\arg \max_j |S_{A_j}| \mid j \in \{1, 2, \dots, |N|\}, k = 5\} \quad (13)$$

Here, F_k represents the set of the top k features ranked by their global SHAP importance.

For Causal Graph Construction (CGC), using the top k features (F_k), we construct a causal graph using the GES (Greedy Equivalence Search) algorithm. Let $C(F_k)$ denote the causal graph:

$$C(F_k) = (V, E) \quad (14)$$

in which $V = F_k$ represents the nodes in the graph, corresponding to the top k features. E represents the directed edges that capture causal relationships inferred by the GES algorithm. Counterfactual Simulations, aiming to evaluate the impact of specific feature changes on the model predictions, we perform counterfactual simulations. Let $\hat{X}$ represent a counterfactual input where the feature j is modified to a new value $\hat{X}_j$. The model's prediction for this modified input is:

$$m(\hat{X}) = m(X \setminus \{X_j\} \cup \{\hat{X}_j\}) \quad (15)$$

Table 4

The classifiers results on the CUSmartHome dataset.

Models	Accuracy			Precision	Recall	F_1 -score	Training time
	Train	Test					
DNN	0.50	0.70	0.70	0.35	0.50	0.41	131.769
Gaussian NB	0.51	0.33	0.33	0.60	0.52	0.29	0.162
Bernoulli NB	0.58	0.41	0.42	0.67	0.58	0.40	0.241
LR	0.50	0.51	0.51	0.51	0.51	0.49	25.829
SGD	0.51	0.32	0.32	0.63	0.51	0.26	63.670
RF	0.99	0.99	100	100	100	100	64.120
XGB	100	100	100	100	100	100	2.798
QDA	0.50	0.67	0.67	0.51	0.50	0.46	0.663
SVC	0.50	0.30	0.31	0.64	0.51	0.25	4.509
DT	0.97	0.96	0.96	0.94	0.97	0.96	0.482

The causal effect of changing feature j is defined as:

$$\Delta_j = m(\hat{X}) - m(X) \quad (16)$$

where Δ_j quantifies the change in the model's prediction caused by modifying feature j .

By integrating SHAP and causal Inference, we achieve a comprehensive interpretability framework:

- **Global Interpretability:** SHAP identifies globally important features (F_k) based on their average contributions.
- The causal graph $C(F_k)$ ensures that these features have causal significance, avoiding spurious correlations.
- **Local Interpretability:** Counterfactual simulations (Δ_j) quantify the impact of feature changes, providing actionable insights.

For example, if a feature X_j has a high SHAP value and a significant causal effect (Δ_j), its importance is both statistical (from SHAP) and causal, reinforcing its reliability for decision-making.

4. The evaluation and the results

In this paper, we used three datasets to ensure the framework's performance, two of which are public. The first set comprised data collected from our Smart Home Lab, and the second is a subset of the IoT23 dataset that is related to our case study Ripple20 vulnerability and its consequences in terms of attacks. The third public dataset is the IoT environment dataset. We tested and evaluated our framework on binary classification and multi-classification. The collected data comprised six IoT devices associated with *Ripple20* vulnerability and six that were clear of this vulnerability. The *IoT23* dataset used consisted of normal packets and DDoS attack packets, while the IoT environment dataset consisted of normal and port scan packets, OS and service detection packets, and flooding attack packets.

First, all these data were per-processed to extract the relevant features. Feature engineering and feature selection were conducted to choose the most relevant features. Then, the data were split into training and testing data, using 80% for training and balancing only these data. Also, to ensure consistency and reproducibility of the splits between different experiments with a fixed random seed $random-state = 42$. Finally, all the data were passed to the framework. It should be noted that our framework can produce the output in each phase that the security expert will see. Performance matrices used to evaluate the model in the data are accuracy for training and testing data, precision, recall, and F1 score.

$$Precision = \frac{TP}{TP + FP} \quad (17)$$

$$Recall = \frac{TP}{TP + FN} \quad (18)$$

$$F_1 = \frac{2 * Precision * Recall}{Precision + Recall} \quad (19)$$

Tables 4–6 show the results of each dataset in the ML engine phase. In addition, we can see the best performance classifier depending on the classification scores and the training time. It can be seen that the XGB classifier consistently demonstrates exceptional accuracy and outstanding performance when detecting vulnerabilities in IoT systems associated with the Ripple20 vulnerability. Thus, our framework selected the XGB classifier as the one with the best performance, which was consequently passed to the XAI phase.

Our proposed XAI framework surpasses the state-of-the-art approaches in terms of accuracy and F1-score on all datasets, as confirmed by the findings of the experiment. Turning to the last output from the final phase of our framework, namely the XAI output, we analyzed the results from the best model, which in our case was XGB, from the ML engine, to clarify and understand the reason for this decision. The use of the SHAP method in XAI analysis provides significant insight into the decision-making process of the top-performing classifier. The analysis of global explanations provides valuable insight into the significance of specific features in the classification task. This process illuminates the underlying patterns and relationships between these features and the detection of vulnerabilities. Including visualizations within the explanation interface dramatically enhances the interpretability of IDS decisions,

Table 5

The classifiers results on the IoT23 dataset.

Models	Accuracy			Precision	Recall	F_1 -score	Training time
	Train	Test					
DNN	0.99	0.99	100	100	100	100	654.301
Gaussian NB	0.99	0.99	100	100	100	100	1.570
Bernoulli NB	0.99	0.99	100	100	100	100	1.430
LR	0.99	0.99	100	100	100	100	128.729
SGD	0.99	0.99	100	100	100	100	42.474
RF	100	100	100	100	100	100	155.310
XGB	100	100	100	100	100	100	15.630
QDA	0.50	0.74	0.74	0.37	0.50	0.43	3.666
SVC	100	0.99	100	100	100	100	33.718
DT	0.99	0.99	100	100	100	100	0.824

Table 6

The classifiers results on the IoT environment dataset.

Models	Accuracy			Precision	Recall	F_1	T_{train}
	Train	Test					
DNN	0.25	0.99	0.10	0.02	0.25	0.05	1042.749
Gaussian NB	0.87	0.93	0.93	0.80	0.88	0.83	1.616
Bernoulli NB	0.31	0.80	0.80	0.28	0.31	0.30	2.686
LR	0.89	0.95	0.96	0.78	0.89	0.81	308.956
SGD	0.75	0.88	0.88	0.72	0.75	0.72	1610.8475
RF	0.99	0.99	0.99	0.97	0.99	0.98	383.896
XGB	0.99	0.99	100	0.96	100	0.98	239.066
QDA	0.43	0.17	0.18	0.48	0.44	0.16	29.764
SVC	0.67	0.94	0.94	0.98	0.67	0.77	122.093
DT	0.92	0.96	0.96	0.84	0.92	0.87	3.923

thereby increasing their accessibility to security experts. The interface facilitates the examination of the importance of the features, the effects of the interaction, and the comprehensive decision-making process, empowering security experts to make well-informed decisions and implement appropriate actions to mitigate vulnerabilities.

The four main outputs from this step in each dataset are, firstly, the most important features that play a significant role in making the decision. These are displayed in terms of feature importance scores, which also take into account the significance of the original features and the decision rules where the features appear. The presented framework, which comprises the SHAP single approach, examines the usage of global explanations to identify the most important features while investigating how they affect the predictions of the final model.

Secondly, demonstrates how changes in a given feature impact the output of the model by displaying and visualizing each feature within the dependence plot to show how changes in this feature affect the model's output. Demonstrates how changes in a given feature impact the output of the model by displaying and visualizing each feature inside the dependent plot. This provides a clearer understanding of the justification behind the classification model's decision. The dependence plots show how the features vary in their impact on the prediction.

Third, to fully comprehend the choice made by the classifier model, visualize only one instance in each dataset using a waterfall plot that shows a single instance explanation of each dataset. The figures demonstrate how SHAP can help us understand the individual predictions made by an ML model, and what features are most important for each instance. The plot has two axes: the x -axis shows the impact of each feature on the model output, and the y -axis shows the feature names and values. The base value $E[f(X)]$ is the expected value of the model output in the training dataset. The final prediction $f(x)$ is the output of the model for this data point. The difference between the base value and the final prediction is the sum of the contributions of each feature. The plot uses different colors to indicate the direction and magnitude of the feature contributions. The red bars represent positive contributions, meaning that the value of the feature increases the output of the model. The blue bars represent negative contributions, meaning that the value of the feature decreases the model output. The length of the bar indicates the absolute value of the contribution. The plot also shows the order of the features according to their importance. The most important feature is the one that has the largest absolute contribution, and it is shown at the top of the plot.

Finally, one of the SHAP outputs, which is the top five important features was passed to causal inference to create two important sources of valuable information and counterfactual simulation. The strengths of CGC include identifying the true reason by eliminating the correlation from causation, enhancing understanding by showing the connections among the variables, confirming the significance of features, and offering helpful resources. Simulated counterfactuals have the advantage of providing insights into how altering certain features affects the results, enables the development and testing of "what if" scenarios, enhances transparency, and helps in decision-making. In general, they offer strong local (counterfactual) and global (CGC) points of view, ensuring accurate and accurate interpretations of the model.

To explore the results in depth in each dataset, the next sections will display and explain the results for each dataset, which are the latest results from our framework.

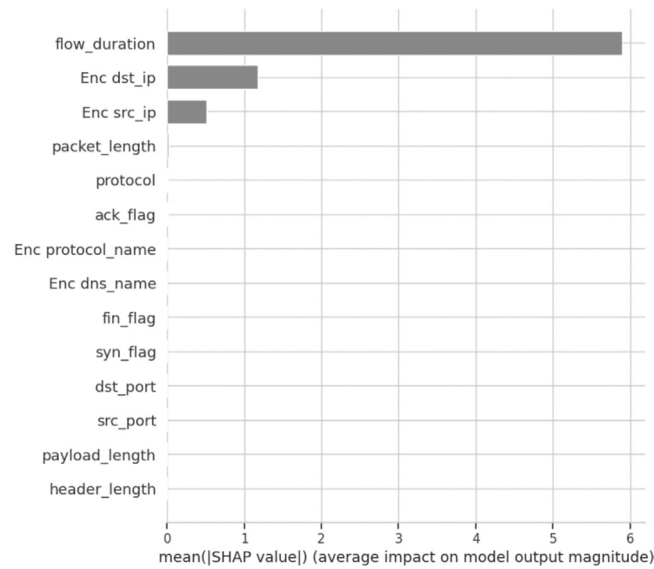


Fig. 3. Feature importance scores in the CUSmartHome dataset.

4.1. CUSmartHome dataset

The first output from the XAI phase comprises the most relevant elements that have a significant influence on the model predictions.¹

Fig. 3 presents the important features that have the highest scores when using SHAP on the collected dataset used to test and evaluate the framework. As we used Ripple20 and its attack detection as a use case in all datasets, the following features were associated with the highest scoring features in the CUSmartHome dataset.

1. The most important feature was *flow-duration*.
2. This was followed by *dst-ip*.
3. The third most important feature was *src-ip*.
4. The fourth feature that affected the model decision was *packet-length*.

The second output from the XAI phase of the proposed framework is that it shows and visualizes each feature within the dependence plot. Some features have strong linear relationships with the prediction, while others have weak or no linear relationships. Some features also interact with other features, while others do not.

In the collected data shown in Fig. 4, the differences can be grouped into three categories. The first category includes features with strong positive linear relationships, namely *payload-length* and *header-length*. These features imply that longer packets and headers significantly increase the SHAP value. Moreover, *payload-length* interacts with *dns-name*, which enhances its effect on prediction. The red points have lower values of *dns-name* and the blue points have higher values of *dns-name*. This suggests that the feature *dns-name* amplifies the effect of the feature *payload-length* on the prediction.

Second, the features with weak or negative linear relationships comprise *packet-length* and *protocol*. This implies that higher values of these features slightly decrease or increase the SHAP value. However, the strength of these relationships varies between the features, with *packet-length* having a stronger relationship than the others. None of these features show clear interactions with their color-coded features, which means that the effect of these features on the prediction is independent of other features.

Third, features with no clear relationships are *src-port*, *dst-port*, *syn-flag*, *ack-flag*, *fin-flag*, *src-ip*, *dst-ip* and *protocol*. This implies that the values of these features do not affect the SHAP value much. These features show random and scattered distributions of the points, which means that there is no pattern or trend in how these features influence the prediction.

The last output from the SHAP phase of the XAI framework is to visualize a single explanation to fully understand the decision behind the classifier model. Fig. 5 shows the influence of each feature on the prediction of the model for a specific instance. The expected output of the model is 0.017 and the actual output for this case is 8.468. The features are ordered by their impact on the prediction, from the most positive to the most negative. The features with positive impact are colored pink and the features with negative impact are colored blue. However, in this case, there are no negative features.

The most influential feature is *flow-duration*, which adds 7.74 to the prediction. This means that the longer the flow duration, the higher the output value. The other two features that have some positive influence are *dst-ip* and *src-ip*, which add 0.39 and 0.35

¹ <https://github.com/CUSmarthome.git>.

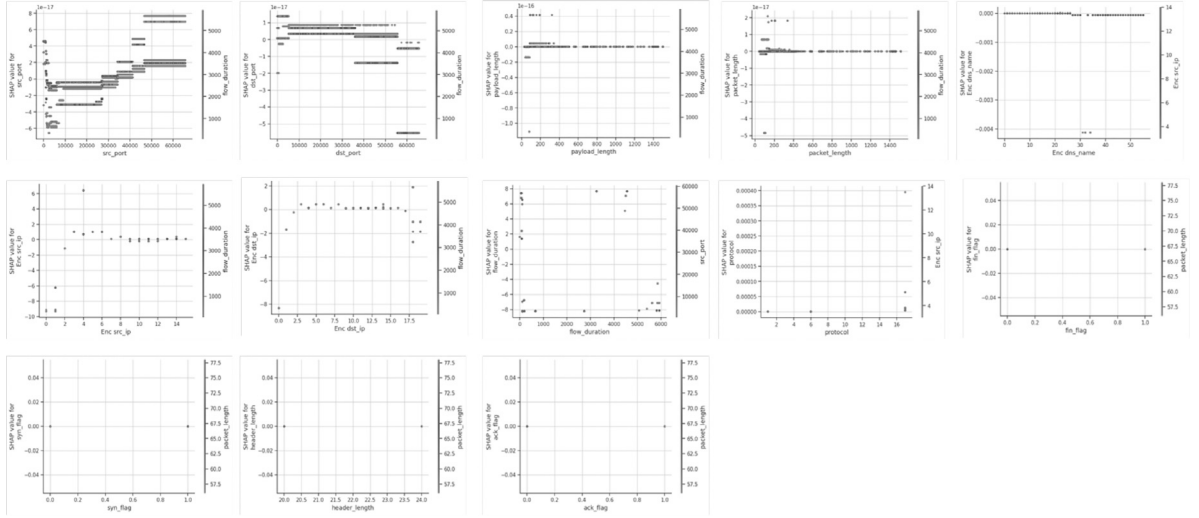


Fig. 4. Causal Analysis for CUSmartHome dataset.

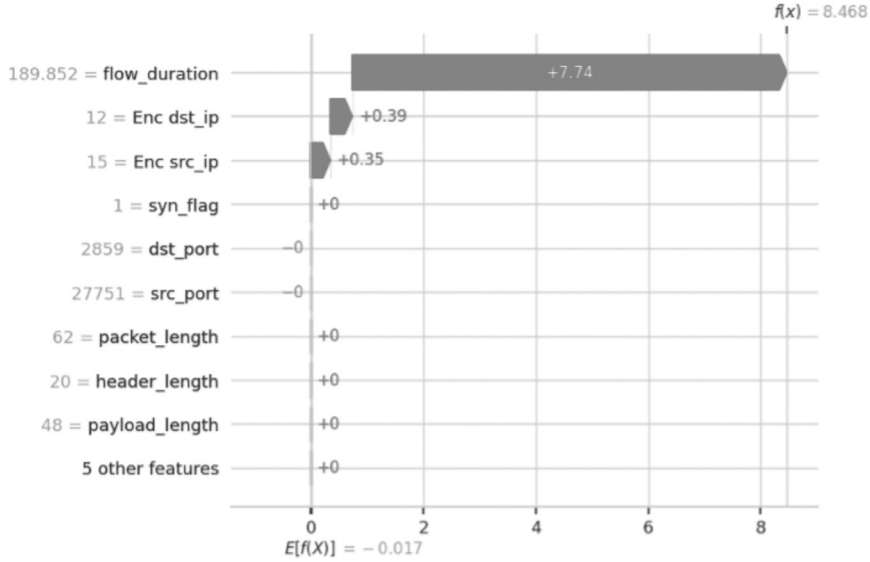


Fig. 5. Visualize a single explanation from CUSmartHome dataset.

to the prediction, respectively. These features represent the encoded destination and source IP addresses. The remaining features do not influence the prediction, as shown by their zero values. These include *syn-flag*, *dst-port*, *src-port*, *packet-length*, *header-length*, *payload-length*, and so on. The figure also shows a dashed line at 8.468, which is the final output value of the model for this instance. This value is the sum of the base value and the feature influences.

By integration between SHAP and Causal Inference, there are two main outputs. Firstly, the CGC figure identifies the relationship between the top five important features that have been calculated by SHAP. Fig. 6 shows how the target variable *label* and the important features of SHAP are related. Significant causal relationships appear that observed factors like *flow-duration* and *payload-length* have a direct and significant impact on the target variable. In addition, the dependencies of the features that appear in the importance of the *src-ip* and *dst-ip* are highlighted by the fact that they are additionally considered causal as they relate to other features.

Secondly, counterfactual simulations in order to evaluate the causal consequences of modifications to certain features, and it was done by changing one feature for one instance and a set of instances, for example in Table 7, we change the *flow-duration* value for a set of instances increased 1.5 times, and as a result, comparisons were made on how the increase in *flow-duration* had effects on the model's predictions. As an example, in quite a few cases, the initial forecast of zero 0.0 was transformed into a counterfactual forecast of 1, hence demonstrating expansion in *flow-duration* made an impact on the decision made by the model. Through this



Fig. 6. Causal graph in the CUSmartHome dataset.

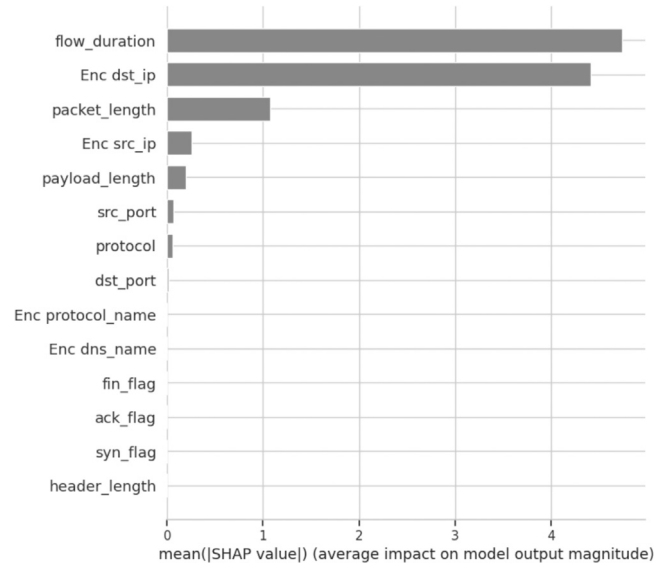


Fig. 7. Feature importance scores in the IoT23 dataset.

simulation, the potential role that the *flow-duration* might have in influencing the model's predictions on the test set, is demonstrated, implying the strength of the feature in the decision-making.

4.2. IoT23 dataset

The initial output of the XAI phase is the most relevant component that significantly affects the model predictions. Fig. 7 presents the important features that have the highest scores when using SHAP on the IoT23 dataset used to test and evaluate the framework. The following features were associated with the highest scoring features in the IoT23 dataset:

1. The most important feature was *flow-duration*.
2. This was followed by *dst-ip*.
3. The third most important feature was *dst-ip*.

Table 7
Counterfactual Simulations on CUSmartHome.

Samples	Original prediction	Counterfactual prediction
0	0	1
1	0	1
2	0	1
3	0	0
4	0	1

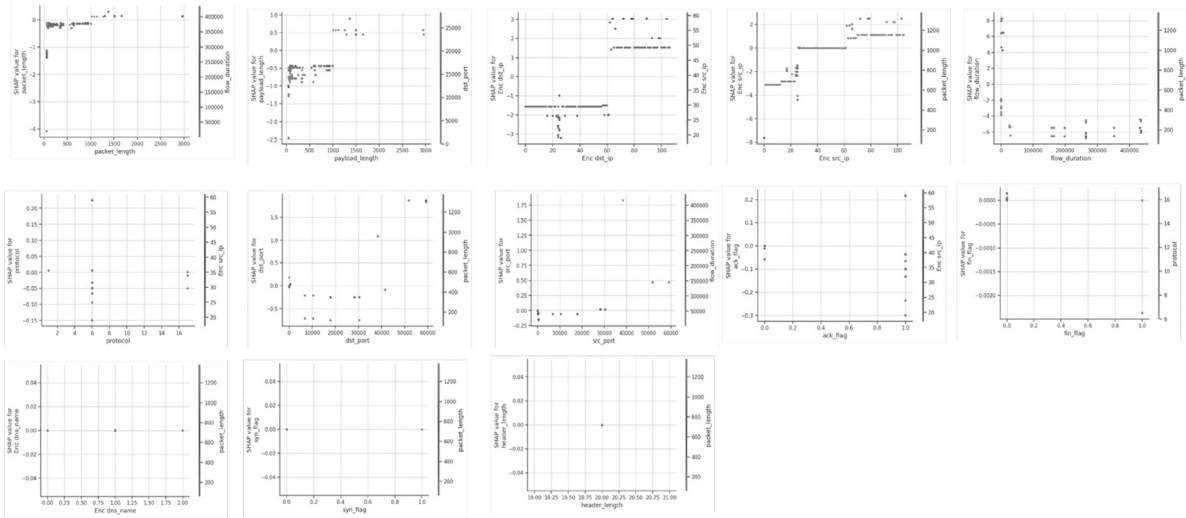


Fig. 8. Features analysis for IoT23 dataset.

4. The fourth feature that affected the model decision was *dst-port*.

The second output of the proposed framework, which comes from the XAI phase, illustrates how every feature within the dependent plot evolves and impacts the model output. While several features have weak or nonexistent linear connections with the prediction, others have robust associations. Although many features do not interact with other features, others do.

In Fig. 8, the relationship between *packet-length* and *flow-duration* shows that there is a distinct pattern where higher packet lengths are associated with shorter *flow-duration*. This could indicate that larger packets result in faster communication flows, while in the relationship between *header-length* and *flow-duration*, the *header-length* shows minimal variation, indicating that it might not have a significant impact on *flow duration*.

In addition, the relationship between *payload-length* and *flow-duration* is similar to *packet length*, as longer payload lengths are associated with shorter flow durations. In addition, the relationship between *protocol* and *flow-duration* indicates that different protocols exhibit varied impacts on flow duration; some protocols are associated with longer durations, while others are not.

The relationships between ports (*src-port* and *dst-port*) and *flow-duration* show that both the source and destination ports have a scattered distribution, suggesting that there is no clear correlation with flow duration. However, the relationship between *syn-flag*, *ack-flag*, *fin-flag* and *flow-duration* indicates that these flags show distinct patterns: for instance, certain flag values (like 1 for the Syn flag) are associated with specific ranges of flow durations.

The final result of the SHAP's is a single explanation that can be visualized to completely comprehend the classifier model's choice. Fig. 9 is a single visualization from the IoT23 dataset. The plot starts from the expected value of the model output in the background data distribution, which is $E[f(X)] = -0.004$, and the final predicted output is -5.764 , which is the sum of all the feature contributions and the expected value. The most influential feature in this prediction is *flow-duration*, which has a positive impact of $+4.98$. This means that the longer the flow duration, the higher the probability that the device is benign. The second most influential feature is *dst-ip*, which has a negative impact on -4.75 . This means that the destination IP address is associated with a higher probability of the device being infected by Ripple20. The third most influential feature is *src-ip*, which also negatively contributes -4.15 . This means that the source IP address is also related to a higher probability of the device being infected by Ripple20. The other features, namely *syn-flag*, *ack-flag*, *fin-flag*, and *dns-name*, have very small positive or negative contributions to the prediction. As a result of the combination of SHAP and Causal Inference, two important results are achieved. Firstly, the CGC figure indicates the causal relationship among the five most important features determined by SHAP. One way to visualize how the target variable, *label*, and the SHAP important features are related is with Fig. 10. For instance, the magnitude of *flow-duration* and *packet-length* on the target variable indicates a substantial direct effect. Also, highlighted are feature dependencies such as *protocol*

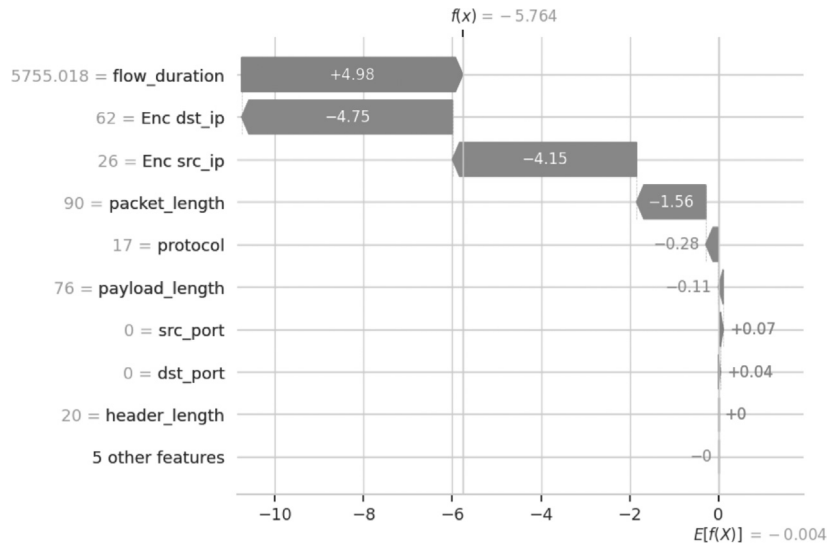


Fig. 9. Visualize a single explanation from IoT23 dataset.

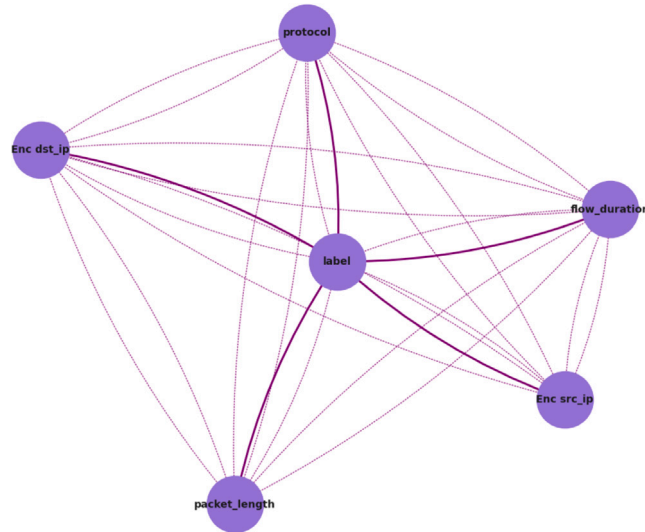


Fig. 10. Causal graph in the IoT23 dataset.

Table 8

Counterfactual simulations on IoT23.

Samples	Original prediction	Counterfactual prediction
0	1	0
1	1	0
2	1	0
3	1	0
4	1	0

and the encoded features *src-ip* and *dst-ip* that reveal their causal relevance based on their interaction with other features. Such insights show intricate interactivity between features that aids in a strong model interpretability and feature selection.

Secondly, counterfactual simulations were performed to assess the causal effect of changing particular features. It did so by modifying one feature for a bunch of instances. In Table 8, for example, the *flow-duration* feature was tripled for several instances. The difference between the original and the counterfactual figure illustrates how this change impacted the model's predictions. As the table clearly shows, in some cases the original prediction of 1 was also changed to a counterfactual prediction of 0, indicating

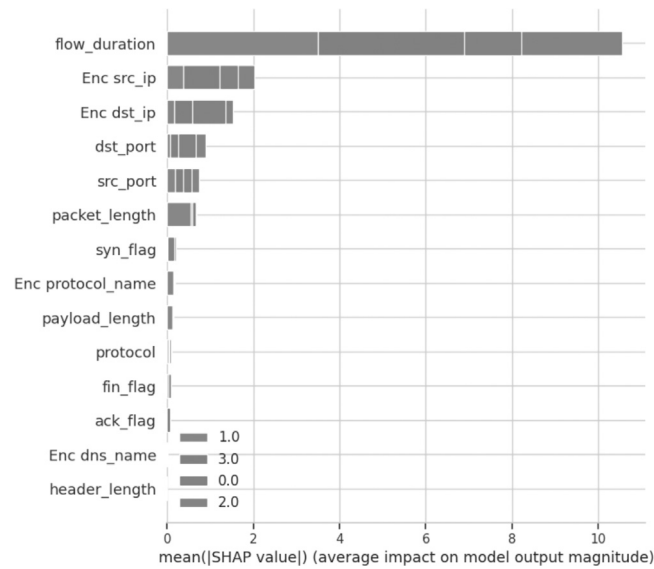


Fig. 11. Feature importance scores in the IoT environment dataset.

that the increase in *flow-duration* impacted the model decision-making process. Indeed, the simulation corroborates how impactful *flow-duration* can be upon the predictions of the model, further making the case for its strength as a feature in the test set (see Fig. 11).

4.3. IoT environment dataset

The first output of the XAI stage shows the most significant features that have a significant impact on the model predictions. The following features were associated with the highest scoring features in the IoT environment dataset:

1. The most important feature was *flow-duration*.
2. This was followed by *src-ip* in IoT environment dataset.
3. The third most important feature was *packet-length*.
4. The fourth feature that affected the model decision was *src-ip*.

The second output of the proposed framework, generated from the XAI phase, shows how each feature in the dependent plot changes over time and influences the model's output. Some features exhibit strong correlations with the prediction, whereas others have weak or nonexistent linear links. Certain features interact with other features, while many do not.

In the IoT environment dataset in Fig. 12, there is a clear distinction in *packet-length* when compared to *header-length*. As *packet-length* increases, there is a significant increase in the SHAP value, indicating a strong positive correlation. This means that *packet-length* is an important feature for model prediction, and higher *packet-lengths* are associated with higher model output. In addition, *Payload-length* shows variability between different *protocols*. Higher *payload-lengths* are associated with specific protocols, suggesting that certain communication protocols handle larger data payloads. This means that *payload-length* and *protocol* are interacting features and that their combination affects the model output differently. Furthermore, *flow duration* exhibits a wide distribution across *source ports*.

Some source ports are associated with longer flow durations, indicating that these ports may be involved in more complex or prolonged communication sessions. This means that *flow-duration* and *source-port* are also interacting features, and their values influence the model output in various ways. Moreover, *destination-port* and *SYN-flag* show an interesting pattern where specific destination ports have higher SYN flag occurrences, pointing towards targeted communication or specific service requests. This means that *destination-port* and *SYN-flag* are another pair of interacting features, and their values have a significant impact on the model output. Finally, the ACK and FIN flags show distinct patterns: higher ACK flags are not necessarily associated with higher FIN flags. This suggests independent behaviors of acknowledgment and session termination within the network traffic. This means that *ACK-flag* and *FIN-flag* are independent features, and their values have different effects on model output. Finally, Fig. 13 shows a single explanation of the IoT environment dataset. The base value $E[f(X)] = 0.465$ is the expected value of the model output in the training dataset we passed. The final prediction $f(x) = 5.764$ is the output of the model for this data point. The most important feature is the one that has the largest absolute contribution, and it is shown at the top of the plot. In this case, the most important feature is *flow-duration*, which has a positive contribution of +3. This means that the value of this feature, 326.698, increases the output of the model by 3 units. The least important feature is the one that has the smallest absolute contribution, and it is shown

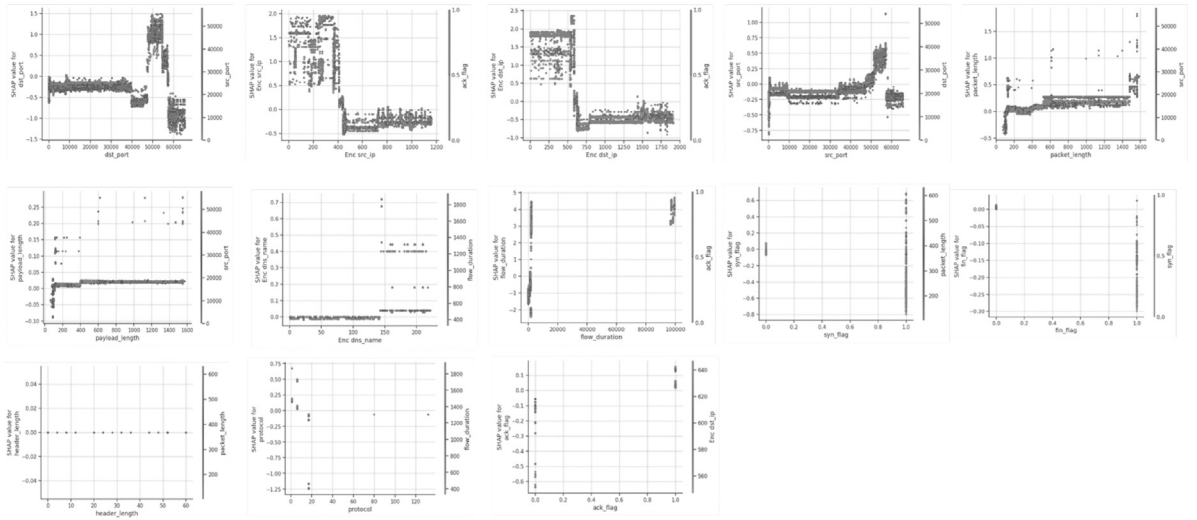


Fig. 12. Features analysis for IoT environment dataset.

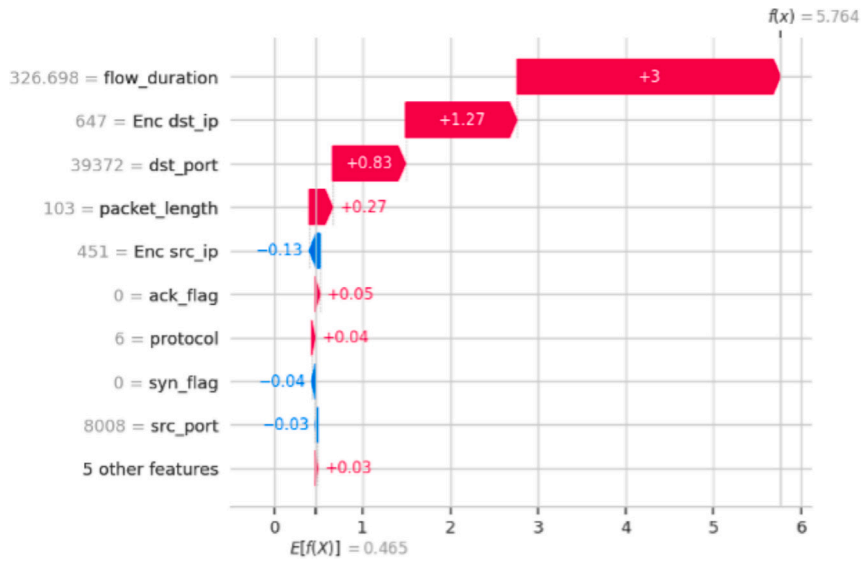


Fig. 13. Visualize a single explanation from IoT environment dataset.

at the bottom of the plot. In this case, the least important features are *fin-flag*, *header-length*, and *dns-name*, which have a positive contribution of +0.03. This means that the combined value of these features increases the output of the model by 0.03 units.

The main two key results generated by the integration of SHAP and Causal Inference: First, the CGC number gives the link between the five most important SHAP value-calculated features As shown in Fig. 14, the predicted variable *label* is linked with the SHAP-important features. Similar to *flow-duration*, *flow-duration* and *packet-length* have strong causal relationships. And dependencies between features are emphasized *src-ip*, *dst-ip* and *dst-port*. These features demonstrate causal relations with the other features, which reinforces their significance. Such insights add to the interpretability of the model, as this captures the significance of features as well as their relevance in a causal sense.

Secondly, counterfactual simulations were performed to quantify the causal impact of changing specific characteristics. This was achieved by modifying a single feature in multiple instances. For example, the *flow-duration* feature was modified for several entries, as detailed in Table 9, to assess its effect. The difference between the original predictions and those that occur in counterfactual predictions shows how these changes impact our model's decisions. In some examples, the original prediction of 3 was instead counterfactually predicted to be 2, and on one it went from an original prediction of 2 to a counterfactual prediction of 0, as shown in the table. These changes show how the *flow-duration* values can make a difference in the prediction of the model decision. This simulation is also critical, as it shows how influential *flow-duration* is when determining predictions, confirming that it is a strong feature of the dataset.

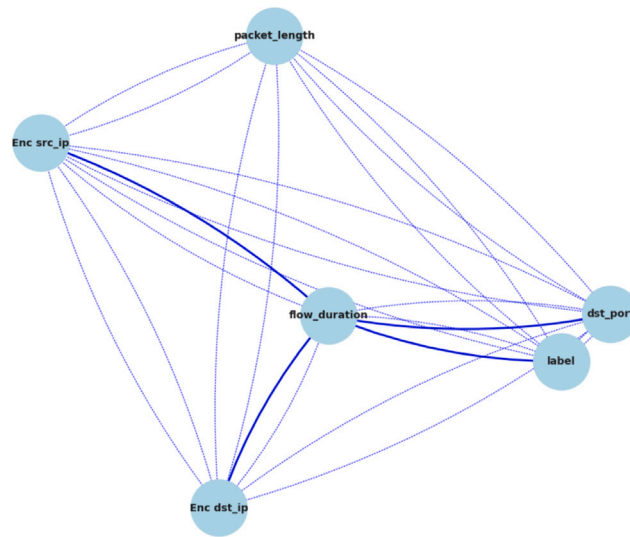


Fig. 14. Causal graph in the IoT environment dataset.

Table 9
Counterfactual simulations on IoT environment dataset.

Samples	Original prediction	Counterfactual prediction
0	3	3
1	3	2
2	3	3
3	3	2
4	2	0

5. Conclusion

This paper introduces a novel Explainable IDS framework for IoT networks that can effectively identify vulnerabilities like Ripple20 and its attacks. The framework combines ML classifiers with XAI embedded with causal inference techniques to provide clear and interpretable explanations for the IDS's decisions. It consists of three stages: data preprocessing, ML engine, and XAI analysis. The ML engine selects the best classifier, achieving high accuracy and performance. XAI techniques such as SHAP with casual inference provide valuable insight into the importance of features and decision-making processes, boosting trust in the system. Extensive experiments on various datasets demonstrate the effectiveness of the framework in identifying and categorizing vulnerable IoT devices. Future work will focus on collecting a more extensive dataset that crosses a wider variety of IoT devices and vulnerabilities, with the aim of increasing the applicability of the framework. The additional major focus will be on improving real-time deployment performance and validating the framework in large-scale IoT network scenarios. Lastly, to add more interpretability and decision-making, progressive explainable techniques and feature selection techniques will be examined. In addition, the goal is to extend the framework to mitigate vulnerabilities and provide interpretable explanations for the best solutions.

CRedit authorship contribution statement

Sarah Bin hulayyil: Writing – original draft, Validation, Methodology, Formal analysis, Conceptualization. **Shancang Li:** Writing – review & editing, Validation, Supervision, Formal analysis, Data curation, Conceptualization. **Neetesh Saxena:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] S. Kumar, P. Tiwari, M. Zymbler, Internet of things is a revolutionary approach for future technology enhancement: A review, *J. Big Data* 6 (1) (2019) 1–21.
- [2] M.K. Hasan, A.A. Habib, Z. Shukur, F. Ibrahim, S. Islam, M.A. Razzaque, Review on cyber-physical and cyber-security system in smart grid: Standards, protocols, constraints, and recommendations, *J. Netw. Comput. Appl.* 209 (2023) 103540.
- [3] M.A. Obaidat, S. Obeidat, J. Holst, A. Al Hayajneh, J. Brown, A comprehensive and systematic survey on the internet of things: Security and privacy challenges, security frameworks, enabling technologies, threats, vulnerabilities and countermeasures, *Computers* 9 (2) (2020) 44.
- [4] S. Latif, W. Boulila, A. Koubaa, Z. Zou, J. Ahmad, Dtl-ids: An optimized intrusion detection framework using deep transfer learning and genetic algorithm, *J. Netw. Comput. Appl.* 221 (2024) 103784.
- [5] P. Anitha, H. Vimala, J. Shreyas, Comprehensive review on congestion detection, alleviation, and control for IoT networks, *J. Netw. Comput. Appl.* 221 (2024) 103749.
- [6] JSOF, JSOF-ripple20, <https://www.jsof-tech.com/disclosures/ripple20/>. (Accessed 13 September 2023).
- [7] Z. Abou El Houda, B. Brik, L. Khoukhi, “Why should i trust your ids?”: An explainable deep learning framework for intrusion detection systems in internet of things networks, *IEEE Open J. Commun. Soc.* 3 (2022) 1164–1176.
- [8] G.D.A. DW, DARPA's explainable artificial intelligence program, *AI Mag.* 40 (2) (2019) 44.
- [9] M. Zolanvari, Z. Yang, K. Khan, R. Jain, N. Meskin, TRUST XAI: A novel model for explainable AI with an example using IIoT security, *IEEE Internet Things J.* (2021).
- [10] S. Tabassum, N. Parvin, N. Hossain, A. Tasnim, R. Rahman, M.I. Hossain, IoT network attack detection using XAI and reliability analysis, in: 2022 25th International Conference on Computer and Information Technology, ICCIT, IEEE, 2022, pp. 176–181.
- [11] Y. Djenouri, A. Belhadi, G. Srivastava, J.C.-W. Lin, When explainable AI meets IoT applications for supervised learning, *Clust. Comput.* 26 (4) (2023) 2313–2323.
- [12] H.A. Abdeljaber, S. Ahmad, A. Alharbi, S. Kumar, XAI-based reinforcement learning approach for text summarization of social IoT-based content, *Secur. Commun. Netw.* 2022 (2022).
- [13] A.I. Gide, A.A. Mu'azu, A real-time intrusion detection system for dos/ddos attack classification in IoT networks using KNN-neural network hybrid technique, *Babylon. J. Internet Things* 2024 (2024) 60–69.
- [14] A. Adadi, M. Berrada, Peeking inside the black-box: A survey on explainable artificial intelligence (XAI), *IEEE Access* 6 (2018) 52138–52160.
- [15] D. Gunning, M. Stefik, J. Choi, T. Miller, S. Stumpf, G.-Z. Yang, XAI—Explainable artificial intelligence, *Sci. Robot.* 4 (37) (2019) eaay7120.
- [16] S.K. Jagatheesaperumal, Q.-V. Pham, R. Ruby, Z. Yang, C. Xu, Z. Zhang, Explainable AI over the internet of things (IoT): Overview, state-of-the-art and future directions, *IEEE Open J. Commun. Soc.* (2022).
- [17] İ. Kök, F.Y. Okay, Ö. Muyanli, S. Özdemir, Explainable artificial intelligence (xai) for internet of things: A survey, *IEEE Internet Things J.* (2023).
- [18] N. Moustafa, N. Koroniotis, M. Keshk, A.Y. Zomaya, Z. Tari, Explainable intrusion detection for cyber defences in the internet of things: Opportunities and solutions, *IEEE Commun. Surv. Tutor.* (2023).
- [19] B.H. Van der Velden, H.J. Kuijff, K.G. Gilhuijs, M.A. Viergever, Explainable artificial intelligence (XAI) in deep learning-based medical image analysis, *Med. Image Anal.* 79 (2022) 102470.
- [20] A. Parmisano, S. Garcia, M. Erquiaga, A Labeled Dataset with Malicious and Benign Iot Network Traffic, Stratosphere Laboratory, Praha, Czech Republic, 2020.
- [21] OCSLab, IoT environment dataset, 2022, <https://ocslab.hksecurity.net/Datasets/iot-environment-dataset>.
- [22] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [23] R. Mohammed, J. Rawashdeh, M. Abdullah, Machine learning with oversampling and undersampling techniques: overview study and experimental results, in: 2020 11th International Conference on Information and Communication Systems, ICICS, IEEE, 2020, pp. 243–248.
- [24] H.-S. Choi, S. Abdirayimov, Demonstrating the power of SHAP values in AI-driven classification of marvel characters, *J. Multimed. Inf. Syst.* 11 (2) (2024) 167–172, <http://dx.doi.org/10.33851/jmis.2024.11.2.167>.
- [25] D. Bowen, L. Ungar, Generalized SHAP: Generating multiple types of explanations in machine learning, 2020, arXiv preprint [arXiv:2006.07155](https://arxiv.org/abs/2006.07155).